smartBridge Internship Program

Credit Card Approval Prediction

Team Detail

Team Members:

1.Dude Bhuvana Sai - 23A21A4429,(CSD)

2.Geetha Rani Adabala - 23A21A4402,(CSD)

3.Guruju Hema Bharagavi - 23A21A4443,(CSD)

4.Jahnavi Thupati -23A21A4498,(CSD)

5.Kaki Sandya - 23A21A4447,(CSD)

College:Swarnandhra College Of Engineering And Technology

Project Description:

Credit Card Approval Prediction System is a machine learning powered web application that automates the initial screening of credit card applicants. Instead of manually reviewing an applicant's financial and personal background, the platform uses a pre-trained Random Forest model to instantly classify an applicant as “Low Risk” (approved) or “High Risk” (denied).

The application is built with Flask and takes user input such as gender, income, education, family status, housing type, age, years employed, and number of family members through a web form. This data is preprocessed on the backend exactly as it was during model training — including converting human-friendly values like “Age” and “Years Employed” back into the model's native “negative days” format, one-hot encoding categorical fields, and reindexing the resulting dataframe to match the model's expected feature columns — before being passed to the model for prediction.

Beyond the core prediction pipeline, the project places strong emphasis on user experience: the original technical, hard-to-use form was redesigned into a clean, modern, glassmorphic interface with friendly language, contextual result messaging, and celebratory confetti animations on approval, turning a purely functional ML demo into a polished, presentable product.

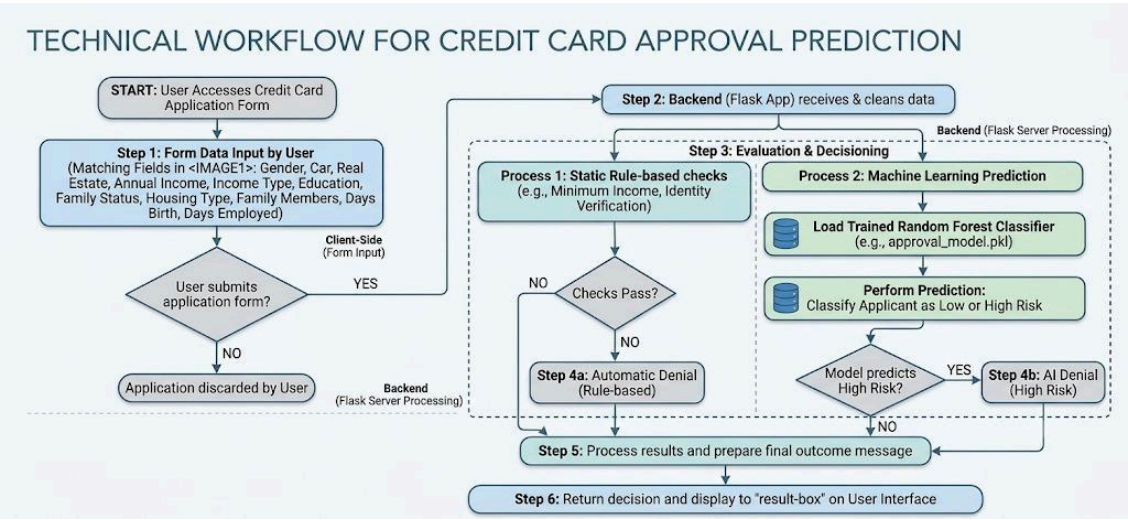
Scenarios

Scenario 1: A salaried applicant enters their age, years employed, income, and marks that they own a house and a car. The system converts age and employment duration into the model's day-based format, encodes the categorical fields, and the Random Forest model classifies them as Low Risk — the result page turns green, displays an encouraging approval message, and confetti animates across the screen.

Scenario 2: A young applicant with a short employment history and no owned property or vehicle submits the form. The model flags the profile as High Risk based on the combination of limited income stability and asset ownership; the result page turns red and shows a supportive “try again” message instead of a harsh rejection.

Scenario 3: An applicant accidentally leaves a required numeric field blank or enters invalid text. The Flask backend catches the exception in the /predict route and re-renders the form with a clear, friendly error message instead of crashing, so the user can correct the input and resubmit.

Technical Architecture:



The application follows a simple three-tier architecture. The frontend is a single HTML page styled with vanilla CSS (glassmorphism cards, gradient background, Inter typography) and JavaScript (canvas-confetti for celebration animations). It collects applicant details and submits them as a standard form POST request. The Flask backend receives the request, converts the raw form data into a pandas DataFrame, applies one-hot encoding and column-reindexing to match the training-time feature set, and passes the processed data to the pre-trained Scikit-Learn Random Forest model loaded from approval_model.pkl. The model's prediction (0 = Low Risk, 1 = High Risk) is mapped to a friendly message and a status flag, both of which are passed back to index.html to drive the dynamic result styling and animations.

Note: *This project does not use a database — the model is trained offline from CSV data (application_record.csv and credit_record.csv) and serialized with pickle, so no ER diagram is applicable.*

Pre-requisites:

- Python Programming Proficiency: Python Documentation
- Flask Framework Knowledge: Flask Documentation
- Data Handling with Pandas & NumPy: Pandas Documentation
- Machine Learning with Scikit-Learn: Scikit-Learn Documentation
- Handling Class Imbalance: imbalanced-learn Documentation
- HTML, CSS, and JavaScript Skills: W3Schools HTML/CSS/JavaScript Tutorials
- Version Control with Git: Git Documentation
- Understanding of the Credit Approval Dataset (application_record.csv & credit_record.csv)

Project Workflow:

- **Milestone 1:** Data Collection, Cleaning & Model Training
  - Activity 1.1: Understand and explore the raw application_record.csv and credit_record.csv datasets.
  - Activity 1.2: Clean the data — handle missing values, remove duplicates, and merge the two datasets.

    - Activity 1.3: Preprocess and engineer features (encode categorical columns, derive a risk label from credit history).
    - Activity 1.4: Train and evaluate a Random Forest classifier, addressing class imbalance with imbalanced-learn.
    - Activity 1.5: Export the trained model as approval_model.pkl using pickle.
- **Milestone 2:** Core Functionalities Development
    - Activity 2.1: Define the application's core features — data capture form, preprocessing pipeline, and prediction display.
    - Activity 2.2: Set up the Flask project structure and load the trained model at startup.
- **Milestone 3:** App.py Development
    - Activity 3.1: Implement the home route and the /predict route, including preprocessing and prediction logic.
- **Milestone 4:** Frontend / UI-UX Development
    - Activity 4.1: Implement UX improvements — human-readable inputs, clear dropdown labels, friendly result messaging.
    - Activity 4.2: Implement UI improvements — glassmorphism design, typography, animations, and confetti effects.
- **Milestone 5:** Deployment & Local Testing
    - Activity 5.1: Set up the environment and install dependencies.
    - Activity 5.2: Run and test the application locally.
- **Milestone 6:** Conclusion

Milestone 1: Data Collection, Cleaning & Model Training

This milestone covers turning the raw Kaggle/UCI-style credit approval data into a trained, deployable model. It focuses on understanding the two source files, cleaning and merging them, engineering the features the model needs, and training a classifier that generalizes well despite the dataset's natural class imbalance (most applicants are not high risk).

Activity 1.1: Understand the Dataset

- application_record.csv: contains applicant demographic and financial fields — gender, car/realty ownership, income, income type, education, family status, housing type, birth date (as days), employment start date (as days), and family member count.
- credit_record.csv: contains each applicant's monthly credit history, used to derive whether an applicant is a good or bad credit risk.
- The two files are linked by a shared applicant ID and were explored in a Jupyter Notebook to understand distributions, missing values, and class balance before any cleaning was done.

Activity 1.2: Data Cleaning

- Handled missing values (notably in OCCUPATION_TYPE and similar sparsely-filled columns).
- Removed duplicate applicant records and clearly invalid or out-of-range entries.
- Merged application_record.csv with the derived risk label from credit_record.csv into a single training dataset.

Activity 1.3: Data Preprocessing & Feature Engineering

- Converted DAYS_BIRTH and DAYS_EMPLOYED (stored as negative day counts) into usable numeric features.
- Applied one-hot encoding (pd.get_dummies()) to categorical columns such as CODE_GENDER, FLAG_OWN_CAR, FLAG_OWN_REALTY, NAME_INCOME_TYPE, NAME_EDUCATION_TYPE, NAME_FAMILY_STATUS, and NAME_HOUSING_TYPE, matching the exact preprocessing the Flask app later replicates at inference time.
- Split the data into training and test sets for model evaluation.

Activity 1.4: Model Training & Selection

- Trained a Random Forest Classifier from Scikit-Learn, chosen for its strong out-of-the-box performance on tabular, mixed-type financial data and its resistance to overfitting relative to a single decision tree.
- Used the imbalanced-learn library to address the natural class imbalance in credit approval data, ensuring the model doesn't simply default to predicting the majority class.
- Evaluated the model on the held-out test set and iterated on preprocessing/hyperparameters to improve reliability.

Activity 1.5: Export the Trained Model

Once satisfied with performance, the model was serialized with pickle so the Flask application could load it without retraining:

```
import pickle
```

```
with open('approval_model.pkl', 'wb') as file:
    pickle.dump(model, file)
```

Milestone 2: Core Functionalities Development

Activity 2.1: Define Core Application Features

- Web Form: captures gender, car/realty ownership, income, income type, education, family status, housing type, age, years employed, and family member count.
- Preprocessing Pipeline: converts human-friendly inputs into the model's native format and aligns them to the model's trained feature columns.
- Prediction & Result Display: runs the model and renders a clear Approved / Denied outcome back to the user.

Activity 2.2: Set Up the Flask Backend

- Initialized the Flask app and enabled TEMPLATES_AUTO_RELOAD so template edits are reflected immediately during development.
- Loaded the trained model once at startup from approval_model.pkl using a relative path, with error handling around the load step so the app fails gracefully if the model file is missing.

```
from flask import Flask, request, render_template
import pickle, pandas as pd, numpy as np, os

app = Flask(__name__)
app.config['TEMPLATES_AUTO_RELOAD'] = True

MODEL_PATH = 'approval_model.pkl'
try:
    with open(MODEL_PATH, 'rb') as file:
        model = pickle.load(file)
    print("Model loaded successfully!")
except Exception as e:
    print(f"Error loading model: {e}")
```

Milestone 3: App.py Development

Milestone 3 covers the two Flask routes that drive the whole application: the home route that serves the form, and the /predict route that does all of the preprocessing and inference work.

Activity 3.1: Writing the Main Application Logic in app.py

- Home route — renders the input form:

```
@app.route('/')
def home():
    return render_template('index.html')
```

- Predict route — captures form data, builds a single-row DataFrame with the exact column names the model was trained on:

```
@app.route('/predict', methods=['POST'])
def predict():
    try:
        # 1. Capture data from the HTML form
        data = {
            'CODE_GENDER': request.form.get('gender'),
            'FLAG_OWN_CAR': request.form.get('car'),
            'FLAG_OWN_REALTY': request.form.get('realty'),
            'AMT_INCOME_TOTAL': float(request.form.get('income')),
            'NAME_INCOME_TYPE': request.form.get('income_type'),
            'NAME_EDUCATION_TYPE': request.form.get('education'),
            'NAME_FAMILY_STATUS': request.form.get('family_status'),
            'NAME_HOUSING_TYPE': request.form.get('housing'),
            'DAYS_BIRTH': -float(request.form.get('age')) * 365.25,
            'DAYS_EMPLOYED': -float(request.form.get('years_employed')) * 365.25,
            'CNT_FAM_MEMBERS': float(request.form.get('family_members'))
        }
```

- Applies the same one-hot encoding used during training, then reindexes the dataframe against model.feature_names_in_ so the inference-time columns line up exactly with the training-time columns, filling any missing dummy columns with 0:

```
        # 2. Convert to DataFrame
        input_df = pd.DataFrame([data])

        # 3. Apply the EXACT same preprocessing used in the notebook
        input_df = pd.get_dummies(input_df)

        # --- Align the web data with the model's expected columns ---
        model_columns = model.feature_names_in_
        input_df = input_df.reindex(columns=model_columns, fill_value=0)
```

- Runs the prediction and maps the numeric output to a friendly message and a status flag used for CSS/animation styling:

```
        # 4. Make prediction
        prediction = model.predict(input_df)

        # 5. Interpret result
        if prediction[0] == 1:
            result = "Oops! You should try again. Application Denied."
            status = "denied"
        else:
            result = "Congratulations! Your application is approved."
            status = "approved"

        return render_template('index.html', prediction_text=result, status=status)

    except Exception as e:
        return render_template('index.html',
                               prediction_text=f"An error occurred: {e}",
                               status="error")
```

- Validate that the request handling covers all three outcomes — approved, denied, and a graceful error message — before moving on to frontend polish.

Milestone 4: Frontend / UI-UX Development

Milestone 4 focuses on transforming a purely functional prediction form into a modern, user-friendly product. The work is split between UX improvements (making the form easier and less error-prone to fill out) and UI improvements (making it visually polished and delightful to use).

Activity 4.1: UX Improvements

- Human-Readable Inputs: instead of asking users to calculate and enter negative-day values (e.g. -12005), the form asks for plain "Age" and "Years Employed"; the backend performs the × -365.25 conversion automatically.
- Clear Dropdown Options: replaced cryptic codes ("F / M", "Y / N") with full words ("Female / Male", "Yes / No") to reduce cognitive load.
- Clearer Messaging: reworded the result text to be friendly and actionable — "Congratulations! Your application is approved." instead of a blunt approve/deny label, and an encouraging "Oops! You should try again." for denials rather than a harsh rejection.

Activity 4.2: UI Improvements

- Modern Design: a glassmorphic card layout on a soft gradient background (#E0E7FF to #F3F4F6).
- Typography: integrated Google Fonts' 'Inter' for clean, modern text rendering.
- Form Styling: soft-bordered inputs, spacious padding, and a blue glow on focus for clear affordance.
- Micro-animations: a hover animation on the submit button and a smooth fade-in for the result box.
- Celebration Animation: integrated the canvas-confetti JavaScript library to burst confetti from the sides of the screen when an application is approved.
- Dynamic Result Styling: the result box changes color based on the prediction — green for approved, red for denied — driven by the status variable passed from Flask.

Milestone 5: Deployment & Local Testing

Milestone 5 covers setting up the environment and running the application locally to verify that the form, preprocessing pipeline, and model prediction all work together end-to-end.

Activity 5.1: Prepare the Environment

- Ensure Python 3.8+ is installed.
- (Recommended) Create and activate a virtual environment:

```
python -m venv venv
.\venv\Scripts\Activate.ps1      # Windows PowerShell
```

- Install the required packages:

```
pip install flask pandas numpy scikit-learn imbalanced-learn
```

- Confirm the project structure matches what app.py expects:

```
credit_card_approval_prediction/
|
|-- app.py                # Main Flask application
```

```
|-- approval_model.pkl    # The trained ML model
|-- templates/
|   `-- index.html        # The web interface
`-- creditcard_csv/
    |-- application_record.csv
    `-- credit_record.csv
```

Activity 5.2: Run and Test the Application

- Start the Flask development server:

```
python app.py
```

- Once running, the terminal will show the local server address:

```
* Running on http://127.0.0.1:5000
```

- Open a browser and navigate to http://127.0.0.1:5000 to access the form.
- Submit sample applicant profiles covering different genders, income levels, and asset ownership combinations to verify that both the “approved” (green, confetti) and “denied” (red) result states render correctly.
- If the browser shows a stale version of the page after a template change, perform a hard refresh (Ctrl+F5 on Windows, Cmd+Shift+R on Mac).

Exploring the Website's Web Pages:

Home / Application Form Page

Description: The single-page interface serves as both landing page and prediction tool. It presents a glassmorphic card on a soft gradient background with the 'Inter' font throughout. The form collects gender, car ownership, property ownership, income, income type, education, family status, housing type, age, years employed, and number of family members, with clear full-word dropdown options and a prominent submit button that includes a subtle hover animation.

Approved Result Page

Description: When the Random Forest model predicts Low Risk, the result box fades in with a green color scheme and the message “Congratulations! Your application is approved.” Canvas-confetti animates bursts of confetti from both sides of the screen to celebrate the outcome.

Denied Result Page

Description: When the model predicts High Risk, the result box fades in with a red color scheme and the friendlier, actionable message “Oops! You should try again. Application Denied.” instead of a blunt rejection, encouraging the user to reconsider their inputs rather than feeling shut out.

Conclusion:

The Credit Card Approval Prediction System successfully combines a trained Random Forest model with a Flask web application to automate credit risk screening in real time. Careful preprocessing — converting human-friendly inputs back into the model's native format, one-hot encoding categorical fields, and reindexing against the model's trained feature columns — ensures that predictions made through the web form are exactly

consistent with how the model was trained. Beyond the ML pipeline, the project's UI/UX overhaul (glassmorphism design, human-readable inputs, friendly messaging, and confetti-driven celebration on approval) demonstrates how a functional data science model can be turned into an approachable, presentable product. Looking ahead, the project could be extended with database-backed application history, user authentication, model explainability (e.g. SHAP values to justify a decision), and richer model evaluation dashboards, positioning it to grow from a prediction demo into a more complete credit-screening tool.